

Promise'i manuel oluşturmak () artık çok az durumda gerekir.
Normalde her şeyi çözer, o yüzden manuel sadece *zorunlu olduğunda* kullanılır.

Aşağıdaki 4 durumda manuel Promise yazılır:

1) Callback kullanan eski fonksiyonları Promise'e çevirmek

Eski JavaScript fonksiyonları callback ile çalışır.
Bunları ile kullanabilmek için manuel Promise gerekir.

Örnek:

2) Kendi zamanlama veya özel asenkron sistemini yazıyorsan

Örneğin animasyon, custom timer, websocket gibi yerde.

3) Promise tabanlı bir kontrol

mekanizması oluşturuyorsan Örneğin dışarıdan tetiklenecek bir “bekleyen söz” (deferred) yapmak.

Bu çok özel durumdur — örneğin bir olayı dışarıdan tetiklemen gerekir.

4) Bir kütüphane yazıyorsan

Bazı durumlarda kütüphaneler düşük seviyeli asenkron kontrol ister.

Mesela:

- özel API wrapper
- socket veya stream yöneticisi
- event sistemi

Bu tarz altyapı işlerinde Promise constructor kullanılır.

Hangi durumlarda asla manuel yazmazsın?

- API isteği ()
- DB isteği
- Dosya okuma

- Normal asenkron işler

Bunlar zaten Promise döndürür → manuel gerekmez.

Özet

Manuel Promise = özel asenkron kontrol gerektiğinde.

Yani “başka yol yoksa” kullanırsın.

✓ **Callback → Promise**

✓ **Kendi timer / event / mekanizma yazıyorsan**

✓ **Promise’in resolve/reject’ini dışarıdan kontrol etmen gerekiyorsa**

✓ **Kütüphane / altyapı kodu yazıyorsan**

Diğer tüm günlük işlerde → **async/await** çok daha modern.

1) “Callback kullanan eski fonksiyonları neden çağırayım?”

Çünkü JavaScript’in içinde **çok eski API’ler**, **kütüphaneler**, **tarayıcı fonksiyonları** ve **Node.js fonksiyonları** hâlâ callback kullanıyor.

Örneğin:

✓ **Node.js’in eski dosya okuma fonksiyonları callback ile çalışıyor**

Ama biz bunu ile kullanmak isteriz → işte burada manuel Promise yazmak gerekir:

✓ Tarayıcıda bazı API'ler callback kullanıyor

Mesela:

-
- Eski
- Bazı medya/animasyon API'leri

Bunları ile kullanmak istiyorsan → Promise constructor gerekir.



2)

nedir?

Bu biraz “ileri seviye” ama sana **çocuk bile anlar şekilde** anlatıyorum:



Bu fonksiyonun yaptığı şey:

“Promise oluştur ama **resolve/reject**’i dışarıda istediğim zaman çalıştırayım.”



Normalde Promise böyle çalışır:

Burada **resolve sadece Promise’in içinde** çağrılabilir.

Ama bazen *Promise’i dışarıdan çözmek istersin.*

Yani şöyle bir şey yapmak istersin:

- Promise dursun, beklesin
- Sonra dışarıdan istediğin bir olay olunca de

Bunu yapmak için özel bir teknik var: **Deferred Promise**.



createDeferred() tam olarak bunu yapıyor.

Kod:

Bu ne anlama geliyor?

- new Promise oluşturuyor
- İçindeki ve değişkenlere aktarılıyor
- Sonra bunları **Promise'in dışına** çıkarıp geri döndürüyor

Yani **resolve** artık dışarıdan çağrılabilir!

Kullanım örneği (çok anlaşılır)

Ne oldu?

1. oluşturuldu
2. Ama resolve içeride değil, dışarıya verildi
3. 2 saniye sonra dışarıdan çağrıldı
4. Promise çözüldü

Neden böyle bir şeye ihtiyaç duyarsın?

Şu durumlarda:

✓ Bir olayı Promise'e çevirmek istiyorsan

Örneğin WebSocket'te "ilk mesaj gelince devam et":

✓ Bir işin tamamlanmasını dışarıdan tetiklemen gerekiyorsa

Animasyon, event, socket, dosya akışı...

✓ Kütüphane veya altyapı yazıyorsan

Bu teknik çok kullanılır.



Sonuç

Callback kullanan fonksiyonları çağırmak zorundasın çünkü:

- Eski API'ler hâlâ kullanılıyor
- Onları await ile kullanmak için Promise'e çevirmen gerekiyor

nedir?

- Promise'in **resolve / reject** fonksiyonlarını dışarı çıkaran bir teknik
- Promise'i **dışarıdan kontrol etmeni** sağlar

Try-catch, JavaScript'te "hata olabilecek işlemleri güvenli şekilde çalıştırmak" için kullanılır.

Ama en çok **asenkron işlemlerde** (fetch, async/await) ve **kontrolü sende olmayan kodlarda** kullanılır.

Aşağıda **frontend'te try-catch'in en çok kullanıldığı alanları** net şekilde sıraladım:

1) API isteklerinde (fetch / axios)

Frontend'de *en sık kullanılan yer burasıdır.*

- ✓ Sunucu çökebilir
- ✓ İnternet olmayabilir
- ✓ Yanlış endpoint olabilir

➡ Hepsini **try-catch yakalar.**

2) JSON parse işlemleri

Backend yanlış veri gönderirse hata fırlatır.

3) Kullanıcı girdisiyle çalışan kodlar

Örn: kullanıcı yanlış format yazarsa.

4) Async/await içinde hatalar

Genelde fonksiyonların içinde try-catch kullanırsın.

5) Tarayıcı API'leri (WebSocket, FileReader, WebRTC, Geolocation)

Bu API'ler tarayıcıya bağımlı olduğu için hata üretme ihtimali yüksektir.

6) Üçüncü parti kütüphaneler

Harici bir kütüphane patlayabilir.

7) Uygulamanın çökmesini engellemek

Özellikle büyük projelerde “uygulama çökmemeli” diye belirli riskli noktaları try-catch ile sararsın.

Kısa özet

Kullanım Alanı	Neden Try-Catch?
API istekleri	En çok hata burada olur
JSON işlemleri	Veri bozuk olabilir
Async fonksiyonlar	await hatayı fırlatır
WebSocket / WebRTC / FileReader	Tarayıcı API'leri kararsız olabilir
Kullanıcı girdisi	Kullanıcı her şeyi yazabilir

İstersen **try-catch** nerede gerekir, nerede gereksizdir onu da anlatayım.

Şunu dedin:

peki her async eklenmesi şartmı

ChatGPT:

Hayır, **her fonksiyona async eklemek kesinlikle şart değil.**

Hatta yanlış yere async eklemek sadece kodu karıştırır.

Aşağıda **async ne zaman gerekli, ne zaman gereksiz** diye çok net bir rehber bıraktım:

1) Eğer fonksiyon içinde kullanıyorsan → **async ŞART**

✓ Çünkü await sadece async fonksiyon içinde çalışır.

2) Fonksiyon zaten Promise döndüren bir şeyi döndürüyorsa → **async GEREKSİZ**

Yanlış kullanım:

Doğru kullanım:

zaten Promise döner → async eklemek **boşa**.

✗ 3) Fonksiyon içinde hiçbir asenkron iş yoksa → async tamamen gereksiz

✗

Doğru:

Async ekleyince fonksiyon **Promise döner**, bu istemediğin bir durum olabilir.

4) Promise manuel oluřturuyorsan → async eklemek ZORUNLU deęil

Burada async yok ünkü:

- ierde await yok
 - direkt Promise dndryorsun
-

5) Bir fonksiyonu “await edilebilir hale getirmek” istiyorsan → async gerekli

Eęer amacın řuyduysa:

- ✓ Bu fonksiyon dıřarıdan await ile kullanılabilirsin

O zaman async mantıklı.

Kısa Özet (çok net)

Async GEREKLİ

3. await kullanıyorsan
 4. fonksiyonunun Promise döndürmesi gerekiyorsa
 5. asenkron işlemleri sıralı yapmak istiyorsan
-

Async GEREKSİZ

4. içinde await yoksa
 5. fonksiyon zaten promise döndüren bir şey döndürüyorsa (fetch, axios)
 6. manuel Promise oluşturuyorsan ama await kullanmıyorsan
 7. sadece değer döndürüyorsan (sync iş)
-

En Önemli Kural:

Async sadece await yazabilmek içindir.
Await yoksa async büyük ihtimalle gereksizdir.